

APOSTILA DE NODE



VISÃO GERAL

Comece por aqui

Node.js leva o JavaScript para além do navegador. Nesta apostila, você vai entender o ambiente de execução, criar um servidor HTTP, conhecer ferramentas do ecossistema e descobrir como a engine V8 transforma código em desempenho.

- 01 Fundamentos do Node.js
- 02 Primeiro servidor web
- 03 Frameworks e ferramentas
- 04 História e evolução
- 05 Instalação e pré-requisitos
- 06 Node.js, browser e V8

Como usar: leia os conceitos, reproduza o exemplo no terminal e use as listas como roteiro de revisão.

Introdução ao Node.js

Node.js é um ambiente de execução JavaScript open source e multiplataforma. É uma ferramenta popular para quase todo tipo de projeto! Node.js roda na engine (motor) V8, o coração do Google Chrome, fora do navegador.

Isso permite que o Node.js seja muito performático. Uma aplicação Node.js roda em um único processo, sem criar uma nova thread para cada requisição. O Node.js provê uma série de primitivas assíncronas para I/O (input/output) em sua biblioteca nativa que previnem códigos JavaScript bloqueantes, e geralmente, bibliotecas em Node.js são escritas usando como padrão paradigmas não-bloqueantes, fazendo com que o comportamento de bloqueio seja uma exceção à regra.

Quando o Node.js executa operações de I/O, como ler dados da rede, acessar um banco de dados ou o sistema de arquivos, em vez de bloquear a thread em execução e gastar ciclos de CPU esperando, o Node.js vai continuar com as operações quando a resposta retornar. Isso permite com que o Node.js lide com centenas de conexões paralelas em um único servidor, removendo o fardo de gerenciar concorrências em threads, que podem ser fontes significativas de bugs. Node.js tem uma vantagem única porque milhões de desenvolvedores frontend que programam JavaScript para o navegador possam agora desenvolver código server-side (backend) além do client-side (frontend), sem a necessidade de aprender uma linguagem completamente diferente.

No Node.js os novos padrões do ECMAScript podem ser utilizados sem problemas, como você não precisa esperar todos os seus usuários atualizarem seus navegadores, você tem a liberdade de decidir qual versão do ECMAScript utilizar, bastando trocar a versão do Node.js, além de poder habilitar especificamente features experimentais, bastando executar o Node.js com as flags apropriadas.

Um número imenso de bibliotecas

O npm (node package manager) com sua simples estrutura ajudou na proliferação do ecossistema Node.js, hospedando atualmente mais de 1,000,000 de pacotes open source que podem ser utilizados gratuitamente.

Um exemplo de aplicação Node.js

O exemplo mais comum de Hello World no Node.js é um servidor web:

```
const http = require('http')
const hostname = '127.0.0.1'
const port = 3000

const server = http.createServer((req, res) => {
  res.statusCode = 200
  res.setHeader('Content-Type', 'text/plain')
  res.end('Hello World\n')
})

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`)
})
```

Para executar esse trecho, salve-o como `server.js` e rode com `node server.js` no seu terminal. Primeiramente esse código importa o módulo `http`. O Node.js tem uma biblioteca padrão fantástica, incluindo suporte de primeira classe para redes.

O método `createServer()` do `http` cria um novo servidor HTTP e o retorna. É definido para o servidor escutar em uma porta e host name (nome de anfitrião, ao pé da letra) específicos. Quando o servidor está pronto, a função `callback` é chamada, nesse caso nos informando que o servidor está rodando.

Sempre que uma nova requisição é recebida, o evento de `request` é chamado, provendo dois objetos: uma requisição (objeto do tipo `http.IncomingMessage`) e uma resposta (objeto do tipo `http.ServerResponse`). Esses 2 objetos são essenciais para manusear a chamada HTTP. O primeiro provê os detalhes da requisição.

Nesse simples exemplo, ele não é utilizado, mas com ele você pode acessar os dados da `request` e as `headers`. O segundo é usado para retornar dados para quem o chamou. Nesse caso com:

```
res.statusCode = 200
```

nós definimos a propriedade `statusCode` como 200, para indicar uma resposta bem sucedida. Nós definimos a `header` de `Content-Type` (tipo de conteúdo):

```
res.setHeader('Content-Type', 'text/plain')
```

e nós fechamos a resposta, adicionando o conteúdo como um argumento do `end()`:

```
res.end('Hello World\n')
```

Frameworks e ferramentas para Node.js

Node.js é uma plataforma de baixo nível. Para facilitar e inspirar os desenvolvedores, milhares de bibliotecas são construídas sob o Node.js pela comunidade. Muitas delas se estabeleceram como opções populares ao decorrer do tempo.

Aqui está uma pequena lista com algumas que valem a pena aprender:

AdonisJs

Um framework full-stack altamente focado na ergonomia, estabilidade e confiança do desenvolvedor. Adonis é um dos frameworks web Node.js mais rápidos.

Express

Provê os meios de se criar servidores web de uma forma muito simples porém poderosa. Tem uma abordagem minimalista, não opinada, focada nas funções essenciais de um servidor, que são a chave do sucesso.

Fastify

Um framework altamente focado em prover a melhor experiência ao desenvolvedor, com o mínimo de gargalos na performance e uma poderosa arquitetura de extensões (plugins). Fastify é um dos mais rápidos frameworks Node.js.

Gatsby

Um gerador de sites estáticos baseado em React, gerido com GraphQL e com um ecossistema de plugins e templates iniciais riquíssimo.

hapi

Um rico framework para construção de aplicações e serviços que habilitam desenvolvedores a focar na escrita das lógicas de aplicação reutilizáveis em vez de perder tempo montando a infraestrutura.

koa

É mantido pelo time por trás do Express, pretende ser ainda menor e simples, construído sob muitos anos de conhecimento. O novo projeto nasceu da necessidade de criar mudanças incompatíveis sem romper com a comunidade existente.

Loopback.io

Faz com que seja fácil construir aplicações modernas que requerem integrações complexas.

Meteor

Um framework full-stack incrivelmente poderoso, permite que você crie aplicações Javascript com uma abordagem isomórfica, compartilhando código entre cliente e servidor. É uma ferramenta generalista que tenta fornecer tudo, que agora integra com bibliotecas frontend como React, Vue, e Angular. Também pode ser utilizado para criar aplicativos mobile.

Micro

Provê um servidor enxuto para criação de microserviços HTTP assíncronos.

NestJS

Um framework Node.js progressivo baseado em Typescript para construção de aplicação de nível empresarial eficientes, confiáveis e escaláveis.

Next.js

Framework React que fornece a melhor experiência de desenvolvimento, com todos os recursos que você precisa para produção: renderização híbrida entre servidor (SSR) e estáticos, suporte ao TypeScript, bundle otimizado, rotas pre-fetching, e mais.

Nx

Um conjunto de ferramentas para desenvolvimento full-stack em monorepo, utilizando NestJS, React, Angular, e mais! Nx ajuda a escalar seu desenvolvimento de um time construindo uma aplicação para múltiplos times colaborando em múltiplas aplicações!

Sapper

Sapper é um framework para construção de aplicações web de todos os tamanhos, com uma bela experiência de desenvolvimento e roteamento flexível baseado em filesystems. Oferece SSR e muito mais!

Socket.io

Um motor de comunicação em tempo real para construir aplicações em rede.

Strapi

Strapi é um CMS flexível, open-source e independente, que fornece aos desenvolvedores a liberdade de escolher seus frameworks e ferramentas favoritos, enquanto também permite que editores administrem e distribuam seus conteúdos de maneira fácil. Por fazer o painel administrativo uma API extensível através de um sistema de plugins, o Strapi permite que as maiores empresas do mundo acelerem a entrega de conteúdo enquanto constroem belas experiências digitais.

Uma breve história do Node.js

Acredite ou não, Node.js só tem 11 anos de idade. Em comparação, o JavaScript tem 24 anos e a Web tem 31 anos. Na tecnologia, 11 anos não é um tempo muito longo, mas o Node.js parece ter existido sempre.

Eu tive o prazer de trabalhar com Node.js desde os primórdios quando ele só tinha 2 anos, e apesar das informações limitadas pela web, você já podia sentir que o crescimento seria enorme.

Nesse post, vamos dissecar a história do Node.js, para colocar as coisas em perspectiva.

Um pouco de história

JavaScript é uma linguagem de programação que foi criada no Netscape como uma ferramenta de scripts para manipulação de páginas web dentro do browser Netscape Navigator. Parte do modelo de negócios da Netscape era vender servidores web, o que incluía um ambiente chamado Netscape LiveWire, que podia criar páginas dinâmicas usando JavaScript no server-side. Infelizmente, o Netscape LiveWire não obteve muito sucesso e JavaScript server-side não era popular até recentemente, com a introdução do Node.js. Um fator chave que levou o Node.js ao topo foi o timing.

Apenas alguns poucos anos antes, Javascript começou a ser mais considerado como uma linguagem séria, graças às aplicações “Web 2.0” (como Flickr, Gmail, etc.) que mostraram ao mundo como poderia ser uma experiência moderna na web. Motores JavaScript (engines) também se tornaram consideravelmente melhores com a competição entre vários browsers para fornecer aos usuários a melhor performance. Times de desenvolvimento por trás dos maiores browsers trabalharam duro para oferecer melhor suporte para o JavaScript e encontrar meios que o fizesse rodar mais rápido.

A engine que roda por baixo dos panos do Node.js, V8 (também conhecida como Chrome V8 por ser a engine Javascript open-source do Projeto Chromium), melhorou significativamente devido a esta competição. Aconteceu de o Node.js ser criado no lugar certo e na hora certa, mas sorte não é a única razão do porquê ser tão popular hoje. Ele introduz várias abordagens e estratégias inovadoras para o desenvolvimento server-side com JavaScript que já ajudaram diversos desenvolvedores.

2009	Nasce o Node.js A primeira forma do npm é criada
2010	Nasce o Express Nasce o Socket.io
2011	npm alcança a versão 1.0 Grandes empresas começam a adotar Node.js: LinkedIn, Uber, etc. Nasce o hapi
2012	Adoção continua muito rápida
2013	Primeira grande plataforma de blogs usando Node.js: Ghost Nasce o Koa
2014	O Grande Fork: io.js é o maior fork do Node.js, com o objetivo de introduzir suporte ao ES6 e crescer rapidamente.
2015	Nasce a Node.js Foundation IO.js é mergeado de volta ao Node.js npm introduz módulos privados Node.js 4 (versões 1, 2 e 3 nunca foram lançadas previamente)
2016	O incidente de leftpad Nasce o Yarn Node.js 6
2017	npm aumenta o foco em segurança Node.js 8 HTTP/2 V8 introduz o Node.js em sua suite de testes, fazendo do Node.js oficialmente um alvo da engine, em adição ao Chrome 3 bilhões de downloads no npm toda semana

2018	Node.js 10 ES modules: suporte experimental ao .mjs Node.js 11
2019	Node.js 12 Node.js 13
2020	Node.js 14 Node.js 15

Como instalar o Node.js

Node.js pode ser instalado de diferentes formas. Esse post destaca as mais comuns e convenientes. Pacotes oficiais para todas as principais plataformas estão disponíveis em <https://nodejs.org/en/download/>.

Uma maneira muito conveniente de instalar o Node.js é através de um gerenciador de pacotes. Neste caso, cada sistema operacional tem sua abordagem mais adequada. No macOS, Homebrew é a alternativa oficial, e - uma vez instalado - permite que a instalação do Node.js seja feita facilmente, rodando esse comando na CLI:

```
brew install node
```

Outros gerenciadores de pacote para Linux e Windows estão listados em <https://nodejs.org/en/download/package-manager/>. nvm é uma maneira muito popular de rodar Node.js. Ele permite que você troque facilmente entre versões, e instale novas versões para testar e troque de volta caso algo pare de funcionar, por exemplo. Também é muito útil para testar seu código com versões antigas do Node.js. Veja <https://github.com/creationix/nvm> para mais informações sobre essa opção.

Minha sugestão é utilizar o instalador oficial se você só estiver começando e não utiliza o Homebrew, caso contrário, o Homebrew é a minha solução favorita. Em todo caso, quando o Node.js é instalado você terá acesso ao comando executável node na linha de comando.

O que devo saber de Javascript para usar o Node.JS?

Como um iniciante, é difícil chegar no ponto onde você é confiante o suficiente nas suas habilidades de programação. Enquanto aprende a programar, você também pode ficar confuso sobre onde o JavaScript termina, e onde o Node.js começa, e vice-versa. Eu recomendo que você tenha um bom domínio dos principais conceitos do JavaScript antes de mergulhar no Node.js:

Fundamentos de JavaScript

- Estrutura Léxica
- Expressões
- Tipos

- Variáveis
- Funções
- this
- Arrow Functions
- Loops
- Escopos
- Arrays
- Template Literals
- Semicolons (;)
- Strict Mode
- ECMAScript 6, 2016, 2017

Com esses conceitos em mente, você está no caminho certo para se tornar um desenvolvedor proficiente em JavaScript, tanto Browser como também Node.js. Os conceitos a seguir também são essenciais para entender programação assíncrona, que é uma parte fundamental do Node.js:

Programação assíncrona

- Programação Assíncrona e callbacks
- Timers
- Promises
- Async e Await
- Closures
- Event Loop

Qual a diferença do Node.JS e do Browser?

Ambos browser e Node.js utilizam JavaScript como sua linguagem de programação. Construir aplicações que rodem no browser é uma coisa completamente diferente de construir aplicações Node.js. Apesar do fato que é sempre JavaScript, há fatores chave que tornam a experiência radicalmente diferente. Da perspectiva de um desenvolvedor frontend que usa JavaScript extensivamente, aplicações Node.js trazem consigo uma enorme vantagem: o conforto de programar tudo - o frontend e o backend - em uma única linguagem.

Você tem uma grande oportunidade porque nós sabemos quão difícil é para aprender, completa e profundamente, uma nova linguagem de programação, e por usar a mesma linguagem para fazer todo o trabalho na web - tanto no servidor quanto no cliente, você está em uma posição

única de vantagem. O que muda é o ecossistema. No browser, na maioria do tempo o que você está fazendo é interagindo com o DOM, ou outras APIs Web como Cookies.

Isso não existe no Node.js, é claro. Você não tem o `document`, `window` e todos os outros objetos que são providos pelo browser. E no browser, nós não temos as APIs legais que o Node.js provê com seus módulos, como a funcionalidade de acesso ao filesystem.

Outra grande diferença é que no Node.js você controla seu ambiente. A não ser que você esteja criando uma aplicação open source que qualquer um pode hospedar em qualquer lugar, você sabe em qual versão do Node.js a aplicação vai rodar. Comparado ao ambiente do browser, onde você não tem o luxo de escolher qual browser seu visitante vai utilizar, isso é muito conveniente.

Isso significa que você pode escrever códigos com os modernos ES6-7-8-9, se atentando ao suporte da sua versão do Node.js. Visto que o JavaScript se move muito rápido, mas os browsers podem ser um pouco lentos para atualizarem, as vezes na web, você está preso em versões velhas do JavaScript / ECMAScript. Você pode utilizar o Babel para transformar seu código em um formato compatível com ES5 antes de enviar para o browser, mas no Node.js, você não precisa disso.

Outra diferença é que o Node.js utiliza o sistema de módulos CommonJS, enquanto que nos browsers ainda estamos vendo o início da implementação do padrão ES Modules. Na prática, isso significa que por enquanto você utiliza `require()` no Node.js e `import` no browser.

O motor V8 do JavaScript

V8 é o nome da engine JavaScript que roda no Google Chrome. É a coisa que pega nosso JavaScript e o executa enquanto navegamos com o Chrome. V8 provê o ambiente de execução em que o JavaScript executa.

O DOM, e outras APIs web são fornecidas pelo browser. O legal é que a engine JavaScript é independente do browser que ela está hospedada. Essa funcionalidade chave possibilitou a ascensão do Node.js.

A V8 foi escolhida como engine por trás do Node.js em 2009, e com a explosão de popularidade do Node.js, a V8 se tornou a engine que agora possibilita uma quantidade incrível de código server-side escrito em JavaScript. O ecossistema Node.js é enorme e isso graças à V8 que também possibilitou aplicações desktop, com projetos como o Electron.

Outras engines JavaScript

Outros browsers têm suas próprias engines:

- Firefox — SpiderMonkey
- Safari — JavaScriptCore (Nitro)
- Edge — Chromium e V8

Todas essas engines implementam o padrão ECMA ES-262, também chamado de ECMAScript, o padrão utilizado pelo JavaScript.

A busca por performance

V8 foi escrita em C++, e é continuamente melhorada. É portátil e roda no Mac, Windows, Linux e diversos outros sistemas. Nessa introdução à V8, vamos ignorar os detalhes de implementação: eles podem ser encontrados em sites mais apropriados (por exemplo, o site oficial da V8), e eles mudam com o passar do tempo, frequentemente.

V8 está sempre evoluindo, assim como as outras engines JavaScript ao seu redor, para agilizar a Web e o ecossistema Node.js. Na web, há uma corrida por performance que vem sendo travada por anos, e nós (como usuários e desenvolvedores) nos beneficiamos muito por essa competição, porque nós possuímos máquinas mais rápidas e otimizadas ano a ano.

Compilação

JavaScript é geralmente considerado como uma linguagem interpretada, mas engines modernas de JavaScript não o interpretam apenas, elas o compilam. Isso vem acontecendo desde 2009, quando o compilador JavaScript SpiderMonkey foi adicionado no Firefox 3.5, e todo mundo seguiu essa ideia. JavaScript é internamente compilado pela v8 com compilação just-in-time (JIT) para acelerar a execução.

Isso pode parecer contra-intuitivo, mas desde a introdução do Google Maps em 2004, o JavaScript evoluiu de uma linguagem que geralmente executava poucas dúzias de linhas de código, para aplicações completas com centenas de milhares de linhas de código executando no browser. Nossas aplicações agora podem rodar por horas dentro do browser, em vez de uma simples validação de regras de formulários ou scripts banais. Nesse novo mundo, compilar JavaScript faz total sentido porque, embora possa demorar um pouco mais para termos o código JavaScript pronto, uma vez concluída a compilação temos muito mais desempenho do que código puramente interpretado.